

CSC 4043 Parallel Computing

Session 4 -Lab Sheet 01

OpenMP - CPU Parallelism
Uses compiler directives (#pragma) to split the work across multiple CPU cores.
Compile with: gcc -fopenmp filename.c -o filename.out Run with: ./filename.out
CUDA - GPU Parallelism
Runs thousands of lightweight threads on NVIDIA GPU cores simultaneously. You write a ‘kernel’ function that each GPU thread executes independently.
Compile with: nvcc filename.cu -o filename.o

Key concepts to Remember:

- Thread: A single unit of parallel execution.
- Reduction: Combining partial results (eg:sum) safely from multiple threads
- Race Condition: All the threads interfere when writing to the same variable. Use reduction, critical or atomic operations.
- Host vs Device: Host is CPU/RAM Device is GPU/VRAM. Data must be copied between them.
- cudaMalloc/ cudaFree: Allocate and release GPU memory.
- cudaMemcpy - Transfer data between CPU and GPU.

Activity 01: Summation

Compute the sum of all elements in the array {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} using

- (a) OpenMP on the CPU
- (b) CUDA on the GPU.

Note: Race Conditions & Safe Reduction

When two threads both execute `sum += value` at the same time, the following happens at the hardware level:

- Thread A reads sum (say, 10).
- Thread B also reads sum (10) before A has written back.
- Both compute `10 + their value` and write their result, one update is lost.

OpenMP fix: `reduction(+:sum)` each thread accumulates into its own private copy; OpenMP adds all copies together safely at the end.

CUDA fix: `atomicAdd(&result, value)` the hardware guarantees that only one thread performs the read-modify-write at a time, so no update is lost.

Activity 02: Matrix Multiplication

Given two $N \times N$ matrices A and B, compute the result matrix C where each element is defined as:

$$C[i][j] = \sum A[i][k] \times B[k][j] \text{ for } k = 0, 1, \dots$$

You may use the identity matrix as B to verify your result.

Matrix A given $A[N][N] = \{\{1, 2, 3, 4\}, \{5, 6, 7, 8\}, \{9, 10, 11, 12\}, \{13, 14, 15, 16\}\}$

Matrix B given $B[N][N] = \{\{1,0,0,0\}, \{0,1,0,0\}, \{0,0,1,0\}, \{0,0,0,1\}\}$;

Note: 2D Parallelism

OpenMP collapse(2): Normally `#pragma omp parallel` for only parallelises the outermost loop. Adding `collapse(2)` merges the i and j loops into a single flat iteration space, giving OpenMP more work to distribute and better thread utilisation.

Why no race condition? Each (i, j) pair is assigned to exactly one thread, which writes exclusively to $C[i][j]$. No two threads ever write to the same cell.

CUDA 2D thread grid: `dim3 blockSize(N, N)` creates a 2D grid where each thread is uniquely identified by `(threadIdx.x, threadIdx.y)`. Each thread computes exactly one output cell — no synchronisation needed.

Flat memory layout: CUDA kernels receive 1D pointers. For an N-column matrix, element $[row][col]$ is stored at index $row * N + col$ this matches how C stores 2D arrays in memory (row-major order).

Activity 03: Finding the maximum value in an Array.

Find the maximum value in the integer array { 3, 1, 4, 1, 5, 9, 2, 6 } using

- (a) OpenMP
- (b) CUDA.

Each thread should scan a portion of the array and report its local maximum; these are then combined to find the global maximum.

Note: Max Reduction

OpenMP reduction(max:max_val): Just like the sum reduction in Activity 1, each thread maintains a private copy of `max_val`. After all threads finish, OpenMP automatically picks the largest value among all private copies.

Important: Initialise `max_val = arr[0]` before the parallel region so the reduction has a valid starting point. Starting the loop from index 1 avoids comparing the first element with itself.

CUDA – atomicMax(&result, value): Like `atomicAdd`, this performs the read-compare-write atomically. `atomicMax` is only defined for int (not float). Copy `arr[0]` to the device result before launching the kernel so threads have a valid initial maximum to compare against.

Activity 04: Vector Addition

Compute the element-wise sum $C = A + B$ for the vectors $A = \{1.0, 2.0, 3.0, 4.0, 5.0, 6.0\}$ and $B = \{6.0, 5.0, 4.0, 3.0, 2.0, 1.0\}$. Every element of C should equal 7.0.

Note: Embarrassingly Parallel Problems

A problem is called embarrassingly parallel when it can be split into completely independent sub-tasks with zero communication between them. Vector addition:

- Each output element $C[i]$ depends only on $A[i]$ and $B[i]$.
- No thread ever reads or writes to another thread's output cell.
- No reduction, no synchronisation, and no atomic operations are needed.
- Performance scales almost linearly: doubling the number of threads roughly halves the runtime.

In CUDA, launch enough threads to cover all N elements. The guard $\text{if } (i < N)$ ensures extra threads (beyond N) do nothing.

Activity 5: Element wise Matrix Addition

Compute the element-wise sum $C = A + B$ for two $N \times N$ matrices where $A[i][j] = i + j + 1$ and $B[i][j] = i - j$. Use $N = 5$.

Note: Extending to 2D

OpenMP `collapse(2)`: Use `collapse(2)` to merge both loops into one flat iteration space (same as Activity 2). This gives OpenMP $N \times N$ independent tasks to distribute far more than N tasks from a single loop.

No race condition: Each thread writes to a unique (i, j) cell, so there is no overlap between threads.

CUDA – flat memory layout: Pass the matrices as 1D arrays. Element $[row][col]$ of an N -column matrix is stored at position $row * N + col$. Use a 2D thread grid (`dim3 blockSize(N, N)`) to assign one thread per cell same pattern as Activity 2.

Activity 6: Parallel Dot Product of Two vectors.

Compute the dot product of $A = \{1.0, 2.0, 3.0, 4.0, 5.0\}$ and $B = \{5.0, 4.0, 3.0, 2.0, 1.0\}$. The dot product is defined as: $\text{dot} = A[0] \times B[0] + A[1] \times B[1] + \dots + A[N-1] \times B[N-1]$.

Note: Combining Multiplication with Reduction

Activity combines two steps:

- Step 1 (map): each thread independently computes its product $A[i] \times B[i]$.
- Step 2 (reduce): all products are safely accumulated into a single total.

This is exactly the same reduction pattern as Activity 1 (array sum), except each thread contributes a product instead of a raw element.

OpenMP: `reduction(+:dot)` handles both steps cleanly. Each thread accumulates $A[i] * B[i]$ into its private copy of `dot`; OpenMP sums all private copies at the end.

CUDA: `atomicAdd(result, A[i] * B[i])` atomically adds each product to the shared result the same atomic accumulation used in Activity 1.

Expected answer: $1 \times 5 + 2 \times 4 + 3 \times 3 + 4 \times 2 + 5 \times 1 = 5 + 8 + 9 + 8 + 5 = 35.0$.

~ End of the Lab Sheet ~